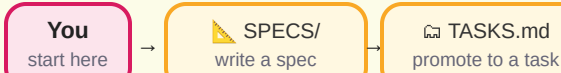


Build Young — Agent Harness (the loop)

You write a spec → it becomes a task; then the loop drains it one at a time: pick & build → self-check → verify → risk gate → ship → record → next. All committed in the repo; a portable playbook is the core; always-on machinery guards every step.

① You author the work — a spec becomes a task



non-trivial → spec it first; a bug-level fix → straight to a task

② Kick off a run — ONE on-ramp

You — run /run-loop locally
your subscription · drains the TASKS.md backlog

GitHub issue + “loop-task” label
→ Actions · a bug-level issue IS the task; else spec it first

☛ then the SessionStart hook (every session):
resync to origin/main · reinstall the commit guards

GitHub repo — single source of truth

all committed here, so a fresh container resumes from it

📁 SPECS/ · 📁 TASKS.md · .github/workflows · src/ — all versioned together

🌐 **ENGINEERING-PLAYBOOK.md**
common & portable across repos — kept in-repo (@imported)

📦 **CLAUDE.md**
this project

📦 **POSITIONING.md**
this project

📦 project docs needn't live anywhere special — best practice: keep them in the repo so the rules travel with the code (versioned, reviewable).

these three are the RULES the doer follows + the verifier grades against.

④ Fan-out — when to go parallel

Default is SEQUENTIAL. Parallelize only if ALL:

- ≥2 tasks on DISJOINT files
 - no foundation-module change mid-flight
 - no dependency · contract (seams) pinnable up front
- sub-agents (worktree+branch) → merge 1-by-1; else seq.
disjointness ← each task's Files: hint + the module map

🌀 /run-loop skill — the driver we built

a Claude Code skill (.claude/skills/run-loop) — the loop's procedure: pick → build → self-check → verify → ship → record.

Run locally (/run-loop) or the Action runs the SAME skill in CI.

Companion /ship = verified merge → PR → squash-merge.

③ The loop — driven by the /run-loop skill (we built it)

Main agent — driver + doer
pick the task (+ read its 📄 spec) · write the smallest change
.claude/skills/run-loop · premium model · stays in the task's file lane

Self-check — gate your own work first
npm run build · npx vitest run · repo guards

Verifier — fresh, independent sub-agent
re-runs build/tests · grades the diff vs criteria + rules
own context (inherits nothing) · model: sonnet · returns PASS / FAIL+gaps

Risk gate
reads the task's risk:

low / med

Ship — on GitHub
commit → push → PR → CI → squash-merge → main

Record — mark [x] in TASKS.md

kicks off the run

the backlog + the rules

the doer may fan out →

drives the loop

📄 The task in play — what the verifier grades against (authored in ① ↩)

each task = goal · acceptance criteria (the verifier grades the diff vs THESE) · risk · committed so a fresh container resumes — its 📄 SPEC holds the full decisions

📖 The rules — read with the task above; the verifier grades every change against both

🌐 **ENGINEERING-PLAYBOOK.md — THE CORE: portable engineering practice (works in any repo)**

Modularity — one feature = one file · foundation vs features · single source of truth · keep the router thin & data-driven

Parallel agents — 4 anti-collision rules: disjoint files · freeze the foundation (additive-only) · contract-first · merge one-at-a-time

Docs as code — CLAUDE.md is the coordination layer · fix stale docs in the same change · every patterned bug becomes a standing rule

Shipping — small verifiable increments · never push to main · build + test + guards before every commit · enforce via hooks/CI

Loop engineering (\$9) — goal + acceptance criteria · doer ≠ verifier split (fresh context) · **model tiering** (premium doer · Sonnet verifier)

· risk drives autonomy (low/med auto-merge; high pauses) · all state in committed files so an ephemeral container resumes

Living architecture (\$3) — the architecture is a committed, auto-regenerated doc with a currency guard (CI blocks a stale one) — this very diagram

📦 **CLAUDE.md — THIS repo's law (project-specific)**

house style (optimize-for-less-scrolling · no flag/emoji glyphs in data · statistics integrity) · the module/endpoint map · quality bars

(WCAG-AA · ~120 KB JS budget · navLock · act() on clickables) · @imports the playbook so the doer auto-loads both

📦 **POSITIONING.md — voice & claims (project-specific)**

tagline “raising builders, not consumers” · what we DON'T claim (no income/return promises; build WITH AI, not coding)

· the canonical phrasings (e.g. “Talk to us” · the First-year builder prize)

Fresh verifier inherits none of this — it is TOLD to read them: 🌐 always · 📦 CLAUDE on app/UI · 📦 POSITIONING on copy · 📄 the task's SPEC when it references one

■ **Pause only on a HARD FLOOR**

an irreversible real-world side-effect → confirm first; else FULL AUTO

e.g. send a real email · charge/refund real money · delete prod data

🛡️ **Always-on machinery — enforced by hooks & GitHub Actions, not memory**

on commit / push / PR → guards + CI block: secrets · internal model-ids · stale diagram (hash) · page-height · push-to-main

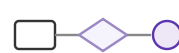
GitHub MCP = the agent's API to GitHub (PR · issue · merge) · git worktrees = fan-out isolation · SessionStart = resync per session

GitHub is the hub: the repo (state) · Issues (an on-ramp) · Actions (run-loop + CI) · Pull Requests (how every change ships)

🌐 **Live site — Vercel**

auto-deploys on squash-merge to main

🌀 **Coordination patterns — same catalog, mapped to our components (◻ agent · ◇ decision · ■ synthesize)**



Classify-And-Act

↳ our risk gate



Fanout-And-Synthesize

↳ fan-out → merge



Adversarial Verification

↳ doer ≠ verifier



Loop Until Done

↳ the loop

Not used (yet): **Generate-And-Filter · Tournament** — one doer per task; quality comes from the verifier + guards, not candidate diversity. Add them (N candidates → filter / pairwise judge) only when a task needs higher-variance solutions.

Same call on coordination: **isolated subagents** (report-back) over **Agent Teams** (inter-agent chat) — orchestrator + serialized merge.